LSGI3245 – Geospatial Database Management and Design

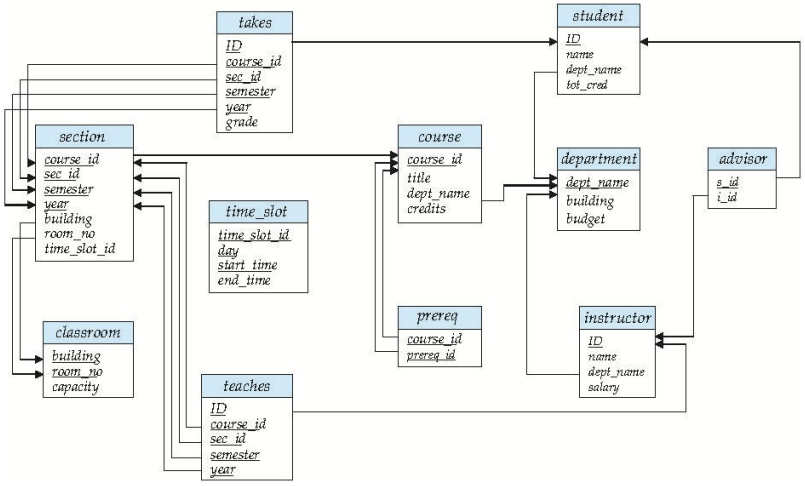
Lecture 4 - SQL (II)

Instructor: Yue YU
Department of Land Surveying and Geo-Informatics,
The Hong Kong Polytechnic University

Tentative Schedule

Week	Lecture (Tuesday)		Lab (Wednesday)	
1	1-Sept	Course Introduction	Lab Introduction and Software Installation	2-Sept
2	8-Sept	Relational Model	Introduction to PostgreSQL (Part 1)	9-Sept
3	15-Sept	SQL (Part 1) Release Information for Final Project	Introduction to PostgreSQL (Part 2)	16-Sept
4	22-Sept	SQL (Part 2)	Query and Data Manipulation Using SQL (Part 1)	23-Sept
5	29-Sept	Intermediate SQL	Query and Data Manipulation Using SQL (Part 2)	30-Sept
6	6-Oct	Database Design	Introduction to PostGIS	7-Oct
7	13-Oct	Spatial Database & PostGIS	PostGIS: Spatial Datatype and Relationships	14-Oct
8	20-Oct	Test 1	PostGIS: Spatial Queries	21-Oct
9	27-Oct	PostGIS & Spatial Queries	Students Work on Final Projects (Part 1)	28-Oct
10	3-Nov	Spatial Storage and Indexing	Students Work on Final Projects (Part 2)	4-Nov
11	10-Nov	Test 2	Students Compile Final Project Reports (Part 1)	11-Nov
12	17-Nov	Project Presentation (Part 1)	Students Compile Final Project Reports (Part 2)	18-Nov
13	24-Nov	Project Presentation (Part 2)	Summary and Outlook	25-Nov

Arrangement



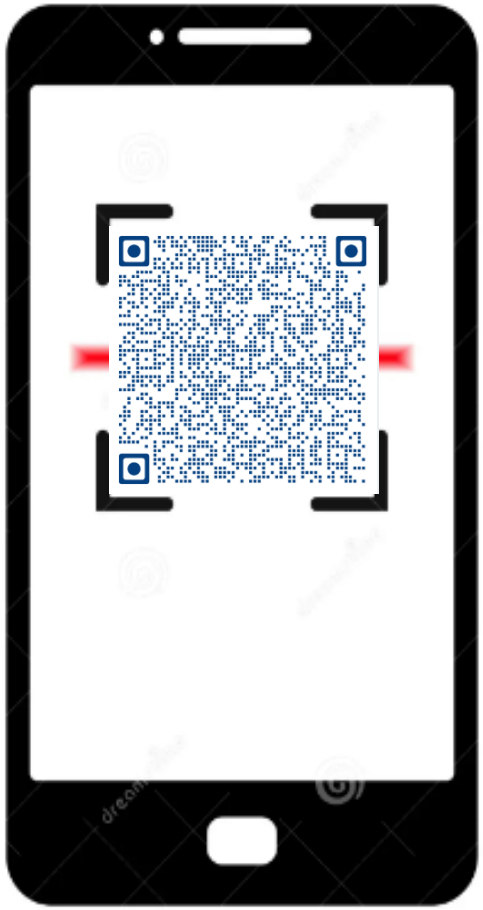
- Section B (Database Operations)
  - Database: Based on the **university database** used in lab.
  - Form: You will be provided with questions that ask you to write down the **SQL queries** that return the correct query result.
  - **Example**: Find the names of all the instructors from the department of 'Biology'

select *name* **from** *instructor* **where** *dept_name*='Biology'

Final Project

Final Project Grouping

- 5 students per group (one as group leader)
  - Note: The assessment is based on the group as an integral part. The contributions of all the group members should be clearly stated in the oral presentation and the final written report, which are also considered for the final assessment of each group member.
- Give the group a good name
- Submit the list of group members via Google Spreadsheet
- **DDL**: by Sep 22, Tuesday (Week 4)
- If you have any queries about grouping, pls contact our TA for help: **Mr. Zhanpeng Zhang** (zhanpeng-2026.zhang@connect.polyu.hk) and **Mr. Feifan Lin** (Feifa.lin@polyu.edu.hk)

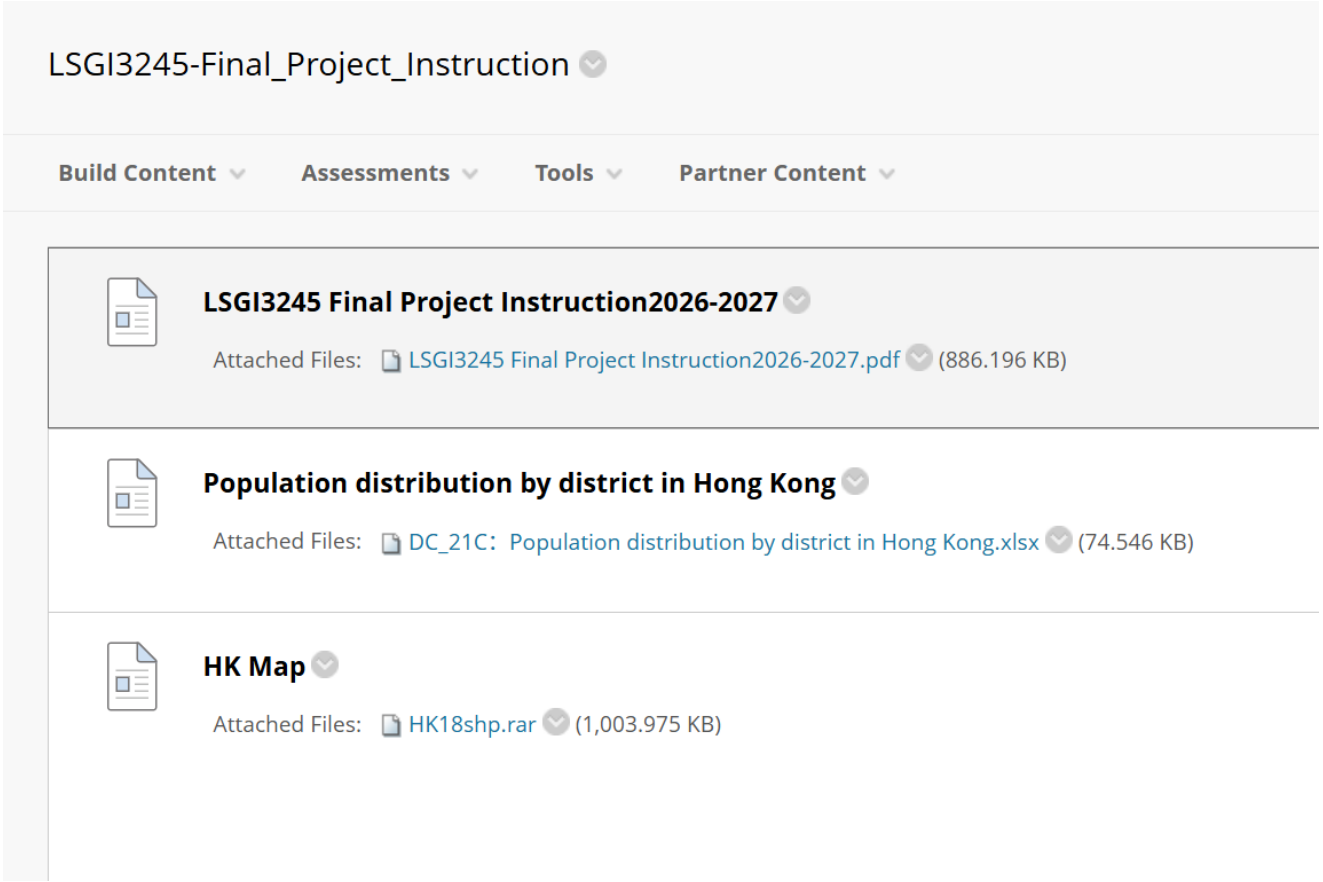


Please Scan the QR code to fill the group information

Overview of Final Project

- Weight: 15%
- Group: 5 students per group
- Deliverables: an oral presentation (10 min/group), a written report (with supplement SQL code)
- Task:
  - To incorporate both spatial and non-spatial information into a database design, and then implement them as a relational database; also demonstrate the mastery of spatial and non-spatial queries/operations over the database created.
- Instruction: Blackboard-Content/LSGI3245-Final_Project_Instruction

Final Project Instruction



Recap



- Understand the basic structure of SQL queries:
  - ***select***, ***from***, ***where***

- Important SQL statements:
  - ***as*** (for rename operation)
  - ***order by, asc, desc*** (for ordering tuples)
  - ***like***, ***ilike*** (for pattern matching)
  - ***union***, ***intersect***, ***except*** (for set operations)
  - ***avg***, ***min***, ***max***, ***sum***, ***count; group by, having*** (for aggregate functions)

Learning Outcomes of Today's Class



- How to use nested **subqueries**?
  - *Subqueries in the where clause*
  - *Subqueries in the from clause*
  - *Subqueries in the with clause*

- Understand the usage of ***natural join***

- Know how to modify a database
  - ***delete***
  - ***insert***
  - ***update***

Nested Subqueries

Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A subquery is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

select A_1, A_2, ..., A_n
from r_1, r_2, ..., r_m
where P

  - A_i can be replaced by a subquery that generates a single value.
  - r_i can be replaced by any valid subquery
  - P can be replaced with an expression of the form:

    B <operation> (subquery)

    Where B is an attribute and <operation> to be defined later.

Subqueries in the where Clause

Subqueries in the where Clause

- A common use of subqueries is to filter results using:
  - Set membership
  - Set comparison
  - Set cardinality

- Remember a query that we have learnt previously by intersecting two different sets:

(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
intersect
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

section
course_id
sec_id
semester
year
building
room_number
time_slot_id

Set Membership

- Find courses offered in Fall 2009 and in Spring 2010:

```
select distinct course_id
from section
where semester = 'Fall' and year = 2009 and
        course_id in (select course_id
                      from section
                      where semester = 'Spring' and year= 2010);
```

(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
intersect
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

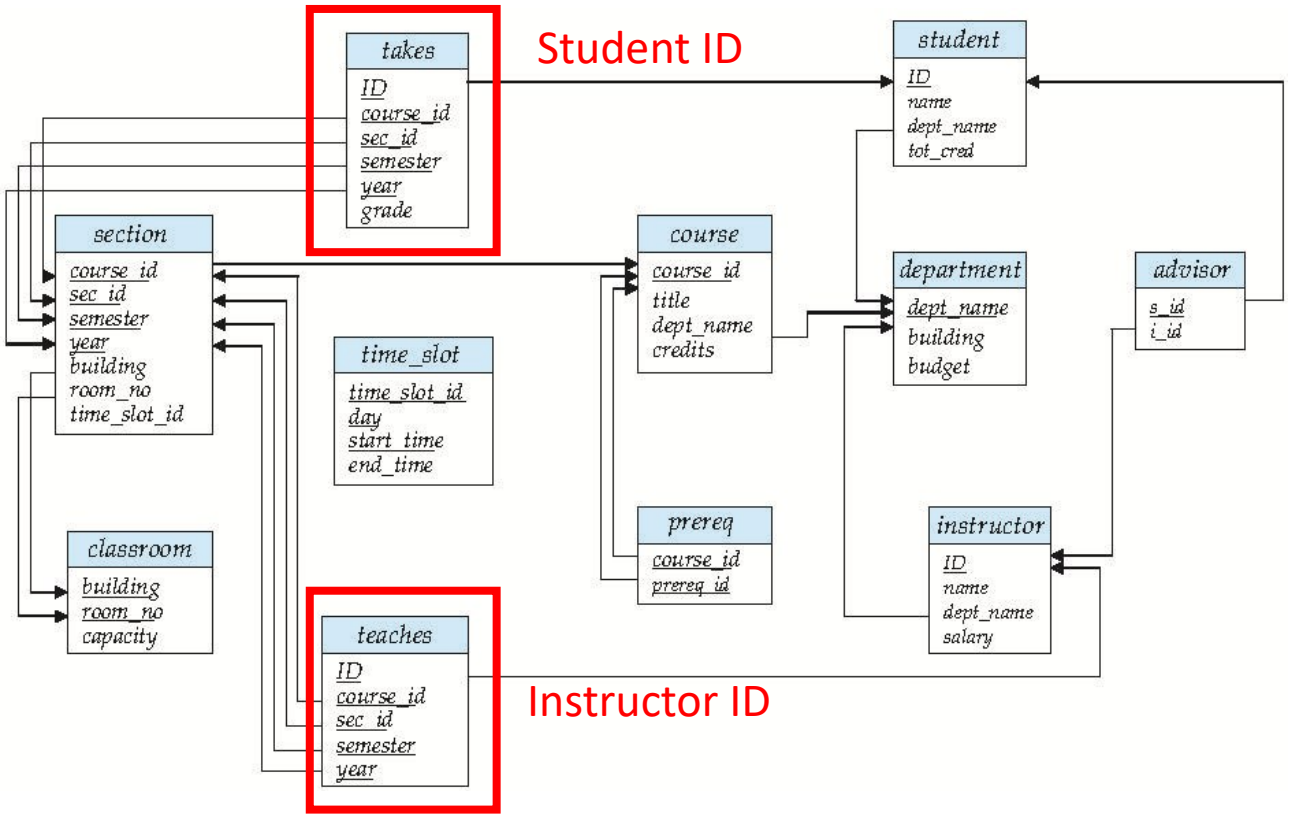
- Find courses offered in Fall 2009 but not in Spring 2010:

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
        course_id  not in (select course_id
                           from section
                           where semester = 'Spring' and year= 2010);
```

(**select** *course_id* **from** *section* **where** *semester* = 'Fall' **and** *year* = 2009)
except
(**select** *course_id* **from** *section* **where** *semester* = 'Spring' **and** *year* = 2010)

Set Membership (Cont.)

- Find the students who have taken course sections taught by the instructor with ID 10101:



Set Membership (Cont.)

- Find the students who have taken course sections taught by the instructor with ID 10101:

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	course_id [PK] character varying (8)	sec_id [PK] character varying (8)	semester [PK] character varying (6)	year [PK] numeric (4)	grade character varying (2)
1	00128	CS-101	1	Fall	2009	A
2	00128	CS-347	1	Fall	2009	A-
3	12345	CS-101	1	Fall	2009	C
4	12345	CS-190	2	Spring	2009	A
5	12345	CS-315	1	Spring	2010	A

Student ID

The **takes** table

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	course_id [PK] character varying (8)	sec_id [PK] character varying (8)	semester [PK] character varying (6)	year [PK] numeric (4)
1	10101	CS-101	1	Fall	2009
2	10101	CS-315	1	Spring	2010
3	10101	CS-347	1	Fall	2009
4	12121	FIN-201	1	Spring	2010

Instructor ID

The **teaches** table

Set Membership (Cont.)

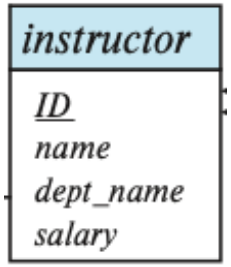
- Find the students who have taken course sections taught by the instructor with ID 10101:

```
select distinct ID
from takes
where (course_id, sec_id, semester, year) in
                    (select course_id, sec_id, semester, year
                     from teaches
                     where teaches.ID = '10101');
```

Set Comparison – *some* Clause

- Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Biology';
```



- The phrase 'greater than at least one' can be represented in SQL by > some.

```
select name
from instructor
where salary > some (select salary
                     from instructor
                     where dept_name = 'Biology');
```

Definition of *some* Clause

- SQL allows ***<some***, ***<=some***, ***>=some***, ***=some***, and ***<>some*** comparisons.

(5 < **some** | 0 / 5 / 6 |) = true (read: 5 < some tuple in the relation)

(5 < **some** | 0 / 5 |) = false

(5 = **some** | 0 / 5 |) = true

(5 ≠ **some** | 0 / 5 |) = true (since 0 ≠ 5)

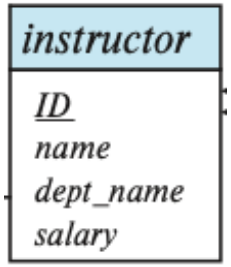
(= **some**) ≡ **in**

However, (≠ **some**) ≢ **not in**

Set Comparison – *all* Clause

- Find the names of all instructors whose salary is greater than the salary of all instructors in the Biology department.

```
select name
from instructor
where salary > all (select salary
                    from instructor
                    where dept_name = 'Biology');
```



Definition of *all* Clause

- As it does for *some*, SQL allows ***<all***, ***<=all***, ***>=all***, ***=all***, and ***<>all*** comparisons.

(5 < **all** | 0, 5, 6 |) = false

(5 < **all** | 6, 10 |) = true

(5 = **all** | 4, 5 |) = false

(5 ≠ **all** | 4, 6 |) = true (since 5 ≠ 4 and 5 ≠ 6)

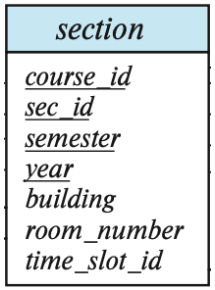
(≠ **all**) ≡ **not in**

However, (= **all**) ≠ **in**

Test for Empty Relations

- SQL includes a feature for testing whether a subquery has any tuples in its result.
- The exists construct evaluates to "true" if a given subquery is nonempty.
- Find courses offered in Fall 2009 and in Spring 2010:

```
select course_id
from section as S
where semester = 'Fall' and year = 2009 and
        exists (select *
                from section as T
                where semester = 'Spring' and year= 2010 and
                        S.course_id = T.course_id);
```



Test for Empty Relations (Cont.)

- Find courses offered in Fall 2009 and in Spring 2010:

Set cardinality

```
select course_id
from section as S
where semester = 'Fall' and year = 2009 and
        exists (select *
                from section as T
                where semester = 'Spring' and year= 2010 and
                        S.course_id = T.course_id);
```

Set membership

```
select distinct course_id
from section
where semester = 'Fall' and year = 2009 and
        course_id in (select course_id
                      from section
                      where semester = 'Spring' and year= 2010);
```

Set operation

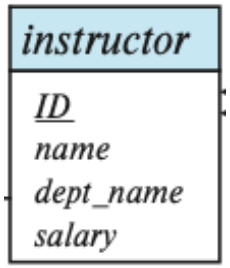
```
(select course_id from section where semester = 'Fall' and year = 2009)
 intersect
(select course_id from section where semester = 'Spring' and year = 2010)
```

Subqueries in the from Clause

Subqueries in the from Clause

- SQL allows a subquery expression to be used in the from clause
- Find the average instructors' salaries of those departments where the average salary is greater than $42,000."

```
select dept_name, avg (salary)
from instructor
group by dept_name
having avg (salary) > 42000;
```




```
select dept_name, avg_salary
from (select dept_name, avg (salary) as avg_salary
      from instructor
      group by dept_name) as T
where T.avg_salary > 42000;
```

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

the **instructor** table

Data Output | Explain | Messages | Query History

	dept_name character varying (20)	avg_salary numeric
1	Biology	72000.000000000000
2	Physics	91000.000000000000
3	Finance	85000.000000000000
4	Music	40000.000000000000
5	History	61000.000000000000
6	Elec. Eng.	80000.000000000000
7	Comp. Sci.	77333.333333333333

select *dept_name*, **avg** (*salary*) **as** *avg_salary*
from *instructor*
group by *dept_name*

Data Output | Explain | Messages | Query History

	dept_name character varying (20)	avg_salary numeric
1	Biology	72000.000000000000
2	Physics	91000.000000000000
3	Finance	85000.000000000000
4	History	61000.000000000000
5	Elec. Eng.	80000.000000000000
6	Comp. Sci.	77333.333333333333

select *dept_name*, *avg_salary*
from (**select** *dept_name*, **avg** (*salary*) **as** *avg_salary*
from *instructor*
group by *dept_name*) **as** *T*
where *T.avg_salary* > 42000;

with Clause

with Clause

- The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the **with** clause occurs.
- Find the department with the highest budget:

```
with max_budget (value) as
        (select max(budget)
         from department)

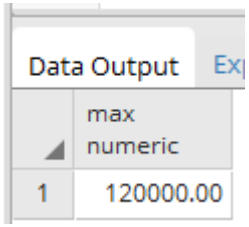
select dept_name
from department, max_budget
where department.budget = max_budget.value;
```



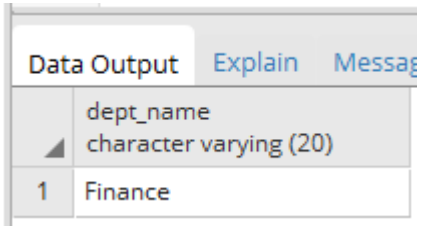
Data Output | Explain | Messages | Query History

	dept_name [PK] character varying (20)	building character varying (15)	budget numeric (12,2)
1	Biology	Watson	90000.00
2	Comp. Sci.	Taylor	100000.00
3	Elec. Eng.	Taylor	85000.00
4	Finance	Painter	120000.00
5	History	Painter	50000.00
6	Music	Packard	80000.00
7	Physics	Watson	70000.00

the **department** table



select max(*budget*)
from *department*




```
with max_budget (value) as
        (select max(budget)
         from department)
select dept_name
from department, max_budget
where department.budget = max_budget.value;
```

Data Output | Explain | Messages | Query History

	dept_name [PK] character varying (20)	building character varying (15)	budget numeric (12,2)
1	Biology	Watson	90000.00
2	Comp. Sci.	Taylor	100000.00
3	Elec. Eng.	Taylor	85000.00
4	Finance	Painter	120000.00
5	History	Painter	50000.00
6	Music	Packard	80000.00
7	Physics	Watson	70000.00

the **department** table

Data Output | Ex

	max numeric
1	120000.00

select max(*budget*)
from *department*

Data Output | Explain | Messag

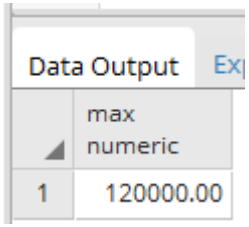
	dept_name character varying (20)
1	Finance

with *max_budget* (*value*) **as**
(**select max**(*budget*)
from *department*)
select *
from *department*, *max_budget*

Data Output | Explain | Messages | Query History

	dept_name [PK] character varying (20)	building character varying (15)	budget numeric (12,2)
1	Biology	Watson	90000.00
2	Comp. Sci.	Taylor	100000.00
3	Elec. Eng.	Taylor	85000.00
4	Finance	Painter	120000.00
5	History	Painter	50000.00
6	Music	Packard	80000.00
7	Physics	Watson	70000.00

the **department** table



select max(*budget*)
from *department*)

```
with max_budget (value) as
        (select max(budget)
         from department)
select *
from department, max_budget
```

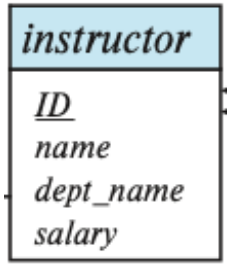
Data Output | Explain | Messages | Query History

	dept_name character varying (20)	building character varying (15)	budget numeric (12,2)	value numeric
1	Biology	Watson	90000.00	120000.00
2	Comp. Sci.	Taylor	100000.00	120000.00
3	Elec. Eng.	Taylor	85000.00	120000.00
4	Finance	Painter	120000.00	120000.00
5	History	Painter	50000.00	120000.00
6	Music	Packard	80000.00	120000.00
7	Physics	Watson	70000.00	120000.00

Complex Queries using with Clause

- Find all departments where the total salary is greater than the average of the total salary at all departments.

```
with dept_total (dept_name, value) as
        (select dept_name, sum(salary)
         from instructor
         group by dept_name),
dept_total_avg(value) as
       (select avg(value)
       from dept_total)
select dept_name
from dept_total, dept_total_avg
where dept_total.value > dept_total_avg.value;
```



```
with dept_total (dept_name, value) as
        (select dept_name, sum(salary)
         from instructor
         group by dept_name),
dept_total_avg(value) as
       (select avg(value)
       from dept_total)
select dept_name
from dept_total, dept_total_avg
where dept_total.value > dept_total_avg.value;
```

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

the **instructor** table

Data Output | Explain | Messages | Query H

	dept_name character varying (20)	sum numeric
1	Biology	72000.00
2	Physics	182000.00
3	Finance	170000.00
4	Music	40000.00
5	History	122000.00
6	Elec. Eng.	80000.00
7	Comp. Sci.	232000.00

select dept_name, **sum**(salary)
from instructor
group by dept_name

Data Output | Explain | Messages | Qu

	avg numeric
1	128285.714285714286

select avg(*value*)
from *dept_total*

What you will get…

```
with dept_total (dept_name, value) as
        (select dept_name, sum(salary)
         from instructor
         group by dept_name),
dept_total_avg(value) as
       (select avg(value)
       from dept_total)
select dept_name
from dept_total, dept_total_avg
where dept_total.value > dept_total_avg.value;
```

Data Output | Explain | Messa

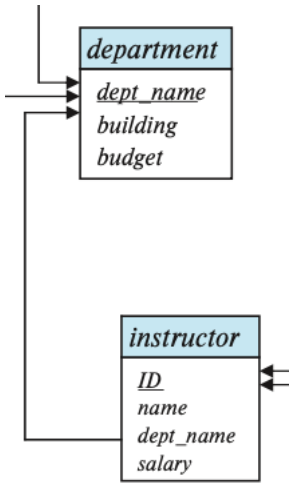
	dept_name character varying (20)
1	Physics
2	Finance
3	Comp. Sci.

Scalar Subquery

Scalar Subquery

- Scalar subquery is one which is used where a single value is expected
- Scalar subqueries can occur in both **select** and **where** clauses
  - List all departments along with the number of instructors in each department

```
select dept_name,
        ( select count(*)
          from instructor
          where department.dept_name = instructor.dept_name)
        as num_instructors
from department;
```



  - Find the department with the highest budget:

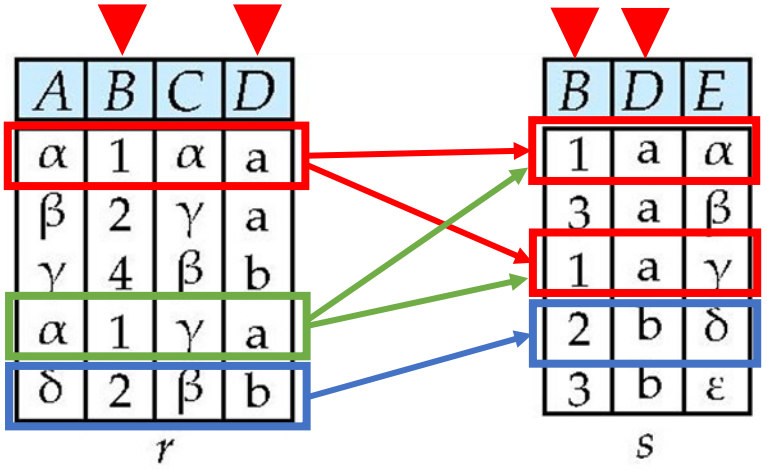
```
select dept_name from department
where department.budget = (select max(budget) from department);
```

Natural Join

Recall: Relational Algebra of Natural Join

- **Natural Join** will give a result that contains the combination of tuples which have the same value on all attributes that have the same name.

Relations r, s:



A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ε

s

Natural Join

$r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

$$\Pi_{A,\, r.B,\, C,\, r.D,\, E}(\sigma_{r.B = s.B \wedge r.D = s.D}(r \times s))$$

The Natural Join

- Recall the following query operation:

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*



- In the above example, the *ID* is the only attribute in the two relations that have the same name.
- To make the life of an SQL programmer easier for this common case, SQL supports an operation called ***natural join***.

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	course_id [PK] character varying (8)	sec_id [PK] character varying (8)	semester [PK] character varying (6)	year [PK] numeric (4)
1	10101	CS-101	1	Fall	2009
2	10101	CS-315	1	Spring	2010
3	10101	CS-347	1	Fall	2009
4	12121	FIN-201	1	Spring	2010
5	15151	MU-199	1	Spring	2010
6	22222	PHY-101	1	Fall	2009
7	32343	HIS-351	1	Spring	2010
8	45565	CS-101	1	Spring	2010
9	45565	CS-319	1	Spring	2010
10	76766	BIO-101	1	Summer	2009
11	76766	BIO-301	1	Summer	2010
12	83821	CS-190	1	Spring	2009
13	83821	CS-190	2	Spring	2009
14	83821	CS-319	2	Spring	2010
15	98345	EE-181	1	Spring	2009

The **teaches** table

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*

Data Output | Explain | Messages | Query History

	name character varying (20)	course_id character varying (8)
1	Srinivasan	CS-101
2	Srinivasan	CS-315
3	Srinivasan	CS-347
4	Wu	FIN-201
5	Mozart	MU-199
6	Einstein	PHY-101
7	El Said	HIS-351
8	Katz	CS-101
9	Katz	CS-319
10	Crick	BIO-101
11	Crick	BIO-301
12	Brandt	CS-190
13	Brandt	CS-190
14	Brandt	CS-319
15	Kim	EE-181

The Natural Join (Cont.)

- Unlike the Cartesian product of the two relations, natural join considers only those pairs of tuples with the same value on those attributes that appear in the schemas of both relations.
- Consider the following two operations:



select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*

select *name, course_id*
from *instructor* **natural join** *teaches*

Are they equivalent?

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	course_id [PK] character varying (8)	sec_id [PK] character varying (8)	semester [PK] character varying (6)	year [PK] numeric (4)
1	10101	CS-101	1	Fall	2009
2	10101	CS-315	1	Spring	2010
3	10101	CS-347	1	Fall	2009
4	12121	FIN-201	1	Spring	2010
5	15151	MU-199	1	Spring	2010
6	22222	PHY-101	1	Fall	2009
7	32343	HIS-351	1	Spring	2010
8	45565	CS-101	1	Spring	2010
9	45565	CS-319	1	Spring	2010
10	76766	BIO-101	1	Summer	2009
11	76766	BIO-301	1	Summer	2010
12	83821	CS-190	1	Spring	2009
13	83821	CS-190	2	Spring	2009
14	83821	CS-319	2	Spring	2010
15	98345	EE-181	1	Spring	2009

The **teaches** table

Data Output | Explain | Messages | Query History

	name character varying (20)	course_id character varying (8)
1	Srinivasan	CS-101
2	Srinivasan	CS-315
3	Srinivasan	CS-347
4	Wu	FIN-201
5	Mozart	MU-199
6	Einstein	PHY-101
7	El Said	HIS-351
8	Katz	CS-101
9	Katz	CS-319
10	Crick	BIO-101
11	Crick	BIO-301
12	Brandt	CS-190
13	Brandt	CS-190
14	Brandt	CS-319
15	Kim	EE-181

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*

select *name, course_id*
from *instructor* ***natural join*** *teaches*

Data Output | Explain | Messages | Query History

	name character varying (20)	course_id character varying (8)
1	Srinivasan	CS-101
2	Srinivasan	CS-315
3	Srinivasan	CS-347
4	Wu	FIN-201
5	Mozart	MU-199
6	Einstein	PHY-101
7	El Said	HIS-351
8	Katz	CS-101
9	Katz	CS-319
10	Crick	BIO-101
11	Crick	BIO-301
12	Brandt	CS-190
13	Brandt	CS-190
14	Brandt	CS-319
15	Kim	EE-181

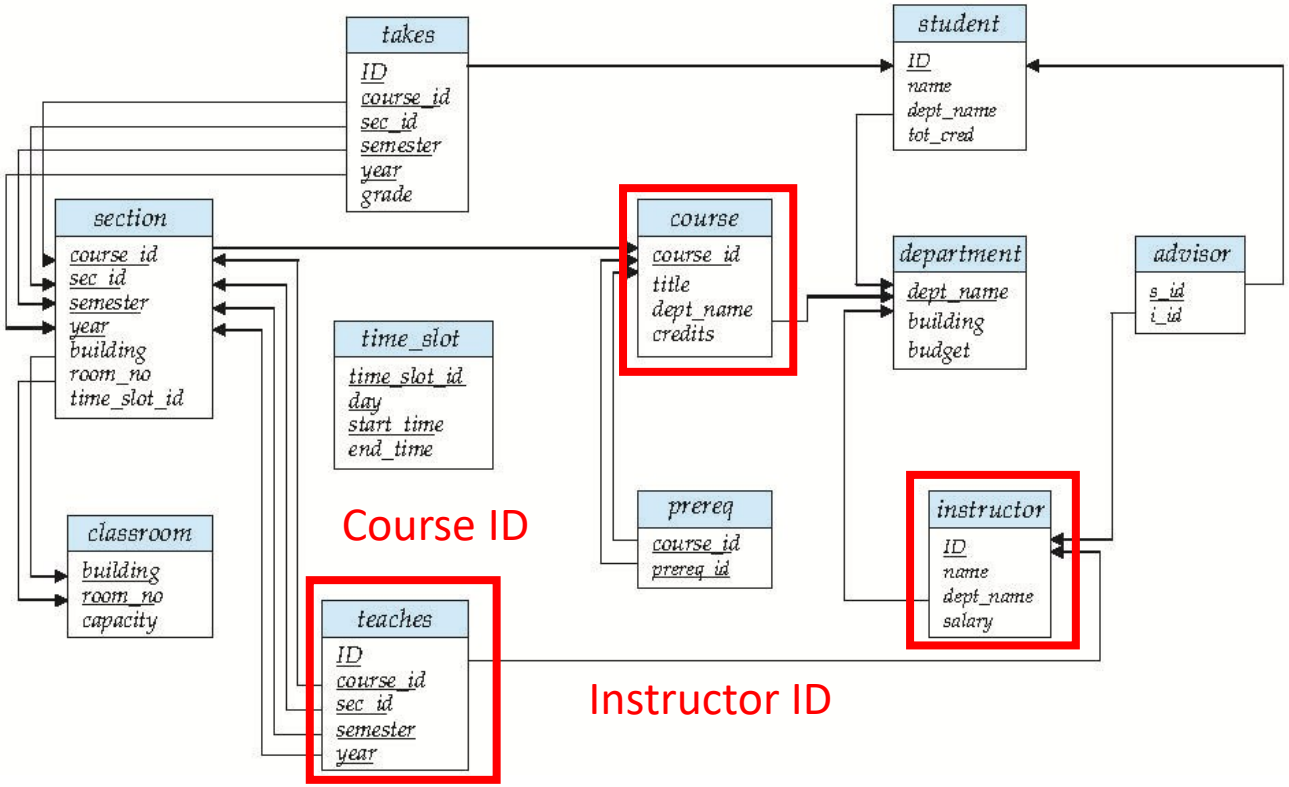
The Natural Join (Cont.)

- A **from** clause in an SQL query can have multiple relations combined using **natural join**:

  select A_1, A_2, …, A_n
  from r_1 **natural join** r_2 **natural join** … **natural join** r_n
  where P

Another Example

- "List the names of instructors along with the titles of courses that they teach".



Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	course_id [PK] character varying (8)	sec_id [PK] character varying (8)	semester [PK] character varying (6)	year [PK] numeric (4)
1	10101	CS-101	1	Fall	2009
2	10101	CS-315	1	Spring	2010
3	10101	CS-347	1	Fall	2009
4	12121	FIN-201	1	Spring	2010
5	15151	MU-199	1	Spring	2010
6	22222	PHY-101	1	Fall	2009
7	32343	HIS-351	1	Spring	2010
8	45565	CS-101	1	Spring	2010
9	45565	CS-319	1	Spring	2010
10	76766	BIO-101	1	Summer	2009
11	76766	BIO-301	1	Summer	2010
12	83821	CS-190	1	Spring	2009
13	83821	CS-190	2	Spring	2009
14	83821	CS-319	2	Spring	2010
15	98345	EE-181	1	Spring	2009

The **teaches** table

Data Output | Explain | Messages | Query History

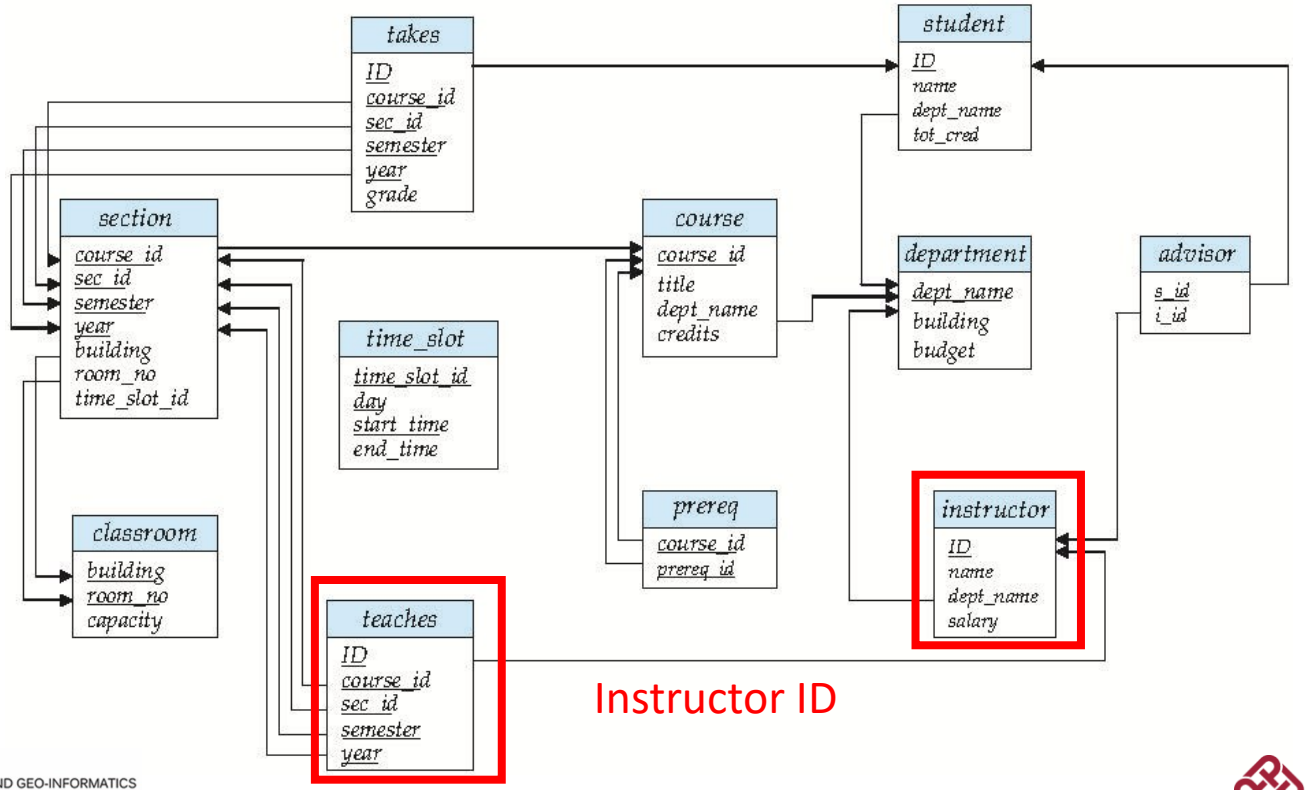
	course_id [PK] character varying (8)	title character varying (50)	dept_name character varying (20)	credits numeric (2)
1	BIO-101	Intro. to Biology	Biology	4
2	BIO-301	Genetics	Biology	4
3	BIO-399	Computational Biology	Biology	3
4	CS-101	Intro. to Computer Science	Comp. Sci.	4
5	CS-190	Game Design	Comp. Sci.	4
6	CS-315	Robotics	Comp. Sci.	3
7	CS-319	Image Processing	Comp. Sci.	3
8	CS-347	Database System Concepts	Comp. Sci.	3
9	EE-181	Intro. to Digital Systems	Elec. Eng.	3
10	FIN-201	Investment Banking	Finance	3
11	HIS-351	World History	History	3
12	MU-199	Music Video Production	Music	3
13	PHY-101	Physical Principles	Physics	4

The **course** table

"List the names of instructors along with the titles of courses that they teach".

Step 1

- "List the names of instructors along with the titles of courses that they teach":
  - Find all the course_ids of the courses that are taught by the instructors.



Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	course_id [PK] character varying (8)	sec_id [PK] character varying (8)	semester [PK] character varying (6)	year [PK] numeric (4)
1	10101	CS-101	1	Fall	2009
2	10101	CS-315	1	Spring	2010
3	10101	CS-347	1	Fall	2009
4	12121	FIN-201	1	Spring	2010
5	15151	MU-199	1	Spring	2010
6	22222	PHY-101	1	Fall	2009
7	32343	HIS-351	1	Spring	2010
8	45565	CS-101	1	Spring	2010
9	45565	CS-319	1	Spring	2010
10	76766	BIO-101	1	Summer	2009
11	76766	BIO-301	1	Summer	2010
12	83821	CS-190	1	Spring	2009
13	83821	CS-190	2	Spring	2009
14	83821	CS-319	2	Spring	2010
15	98345	EE-181	1	Spring	2009

The **teaches** table

Data Output | Explain | Messages | Query History

	name character varying (20)	course_id character varying (8)
1	Srinivasan	CS-101
2	Srinivasan	CS-315
3	Srinivasan	CS-347
4	Wu	FIN-201
5	Mozart	MU-199
6	Einstein	PHY-101
7	El Said	HIS-351
8	Katz	CS-101
9	Katz	CS-319
10	Crick	BIO-101
11	Crick	BIO-301
12	Brandt	CS-190
13	Brandt	CS-190
14	Brandt	CS-319
15	Kim	EE-181

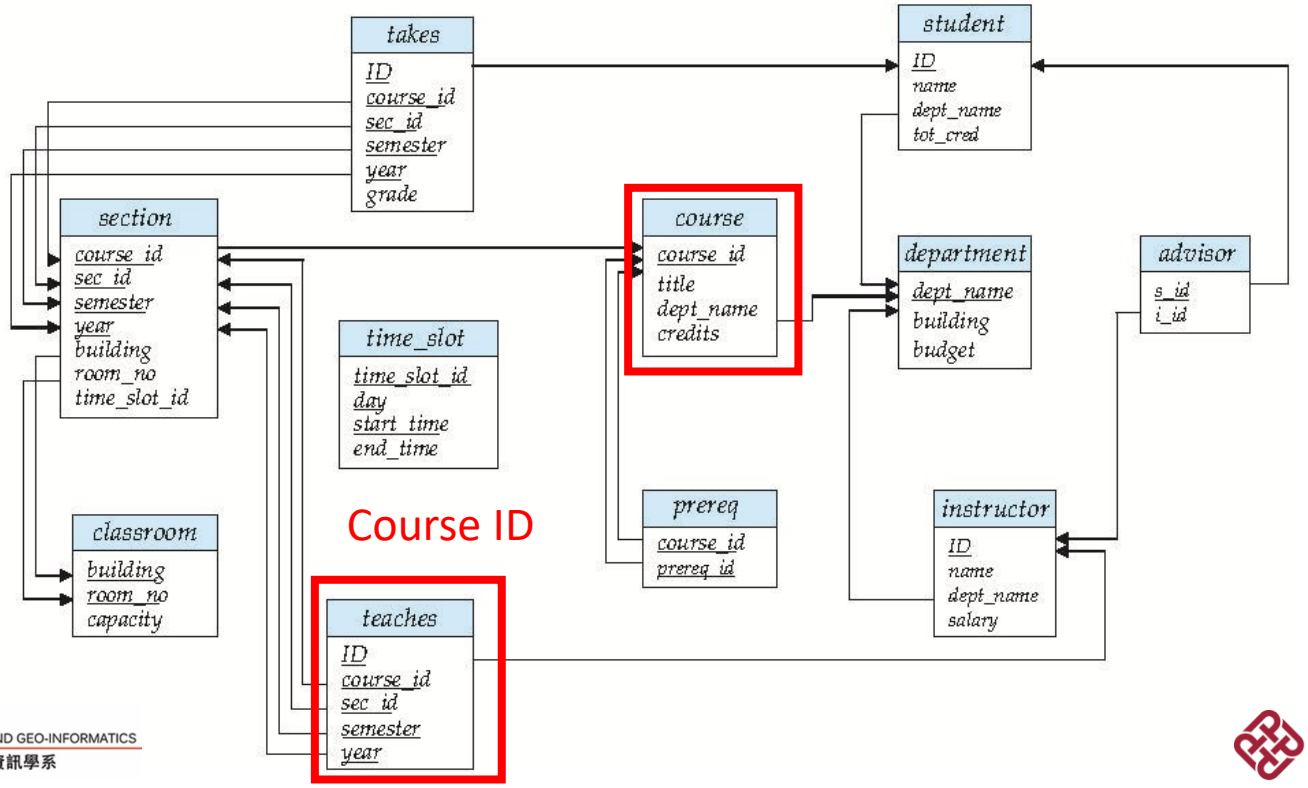
select *name, course_id*
from *instructor* ***natural join*** *teaches*

Equivalent to:

select *name, course_id*
from *instructor, teaches*
where *instructor.ID = teaches.ID*

Step 2

- "List the names of instructors along with the titles of courses that they teach":
  - For those course_ids that have been located, we need to find their names in the **course** table.



Data Output | Explain | Messages | Query History

	name character varying (20)	course_id character varying (8)
1	Srinivasan	CS-101
2	Srinivasan	CS-315
3	Srinivasan	CS-347
4	Wu	FIN-201
5	Mozart	MU-199
6	Einstein	PHY-101
7	El Said	HIS-351
8	Katz	CS-101
9	Katz	CS-319
10	Crick	BIO-101
11	Crick	BIO-301
12	Brandt	CS-190
13	Brandt	CS-190
14	Brandt	CS-319
15	Kim	EE-181

select *name, course_id*
from *instructor* **natural join** *teaches*

Data Output | Explain | Messages | Query History

	course_id [PK] character varying (8)	title character varying (50)	dept_name character varying (20)	credits numeric (2)
1	BIO-101	Intro. to Biology	Biology	4
2	BIO-301	Genetics	Biology	4
3	BIO-399	Computational Biology	Biology	3
4	CS-101	Intro. to Computer Science	Comp. Sci.	4
5	CS-190	Game Design	Comp. Sci.	4
6	CS-315	Robotics	Comp. Sci.	3
7	CS-319	Image Processing	Comp. Sci.	3
8	CS-347	Database System Concepts	Comp. Sci.	3
9	EE-181	Intro. to Digital Systems	Elec. Eng.	3
10	FIN-201	Investment Banking	Finance	3
11	HIS-351	World History	History	3
12	MU-199	Music Video Production	Music	3
13	PHY-101	Physical Principles	Physics	4

The **course** table

What should we do here?

Data Output | Explain | Messages | Query History

	name character varying (20)	course_id character varying (8)
1	Srinivasan	CS-101
2	Srinivasan	CS-315
3	Srinivasan	CS-347
4	Wu	FIN-201
5	Mozart	MU-199
6	Einstein	PHY-101
7	El Said	HIS-351
8	Katz	CS-101
9	Katz	CS-319
10	Crick	BIO-101
11	Crick	BIO-301
12	Brandt	CS-190
13	Brandt	CS-190
14	Brandt	CS-319
15	Kim	EE-181

select *name, course_id*
from *instructor* **natural join** *teaches*

Data Output | Explain | Messages | Query History

	course_id [PK] character varying (8)	title character varying (50)	dept_name character varying (20)	credits numeric (2)
1	BIO-101	Intro. to Biology	Biology	4
2	BIO-301	Genetics	Biology	4
3	BIO-399	Computational Biology	Biology	3
4	CS-101	Intro. to Computer Science	Comp. Sci.	4
5	CS-190	Game Design	Comp. Sci.	4
6	CS-315	Robotics	Comp. Sci.	3
7	CS-319	Image Processing	Comp. Sci.	3
8	CS-347	Database System Concepts	Comp. Sci.	3
9	EE-181	Intro. to Digital Systems	Elec. Eng.	3
10	FIN-201	Investment Banking	Finance	3
11	HIS-351	World History	History	3
12	MU-199	Music Video Production	Music	3
13	PHY-101	Physical Principles	Physics	4

The **course** table

select *name, title*
from *instructor* ***natural join*** *teaches, course*
where *teaches.course_id = course.course_id*

Result

Data Output | Explain | Messages | Query History

	name character varying (20)	title character varying (50)
1	Srinivasan	Intro. to Computer Science
2	Srinivasan	Robotics
3	Srinivasan	Database System Concepts
4	Wu	Investment Banking
5	Mozart	Music Video Production
6	Einstein	Physical Principles
7	El Said	World History
8	Katz	Intro. to Computer Science
9	Katz	Image Processing
10	Crick	Intro. to Biology
11	Crick	Genetics
12	Brandt	Game Design
13	Brandt	Game Design
14	Brandt	Image Processing
15	Kim	Intro. to Digital Systems

Perform the ***Natural Join***

Data Output | Explain | Messages | Query History

	name character varying (20)	course_id character varying (8)
1	Srinivasan	CS-101
2	Srinivasan	CS-315
3	Srinivasan	CS-347
4	Wu	FIN-201
5	Mozart	MU-199
6	Einstein	PHY-101
7	El Said	HIS-351
8	Katz	CS-101
9	Katz	CS-319
10	Crick	BIO-101
11	Crick	BIO-301
12	Brandt	CS-190
13	Brandt	CS-190
14	Brandt	CS-319
15	Kim	EE-181

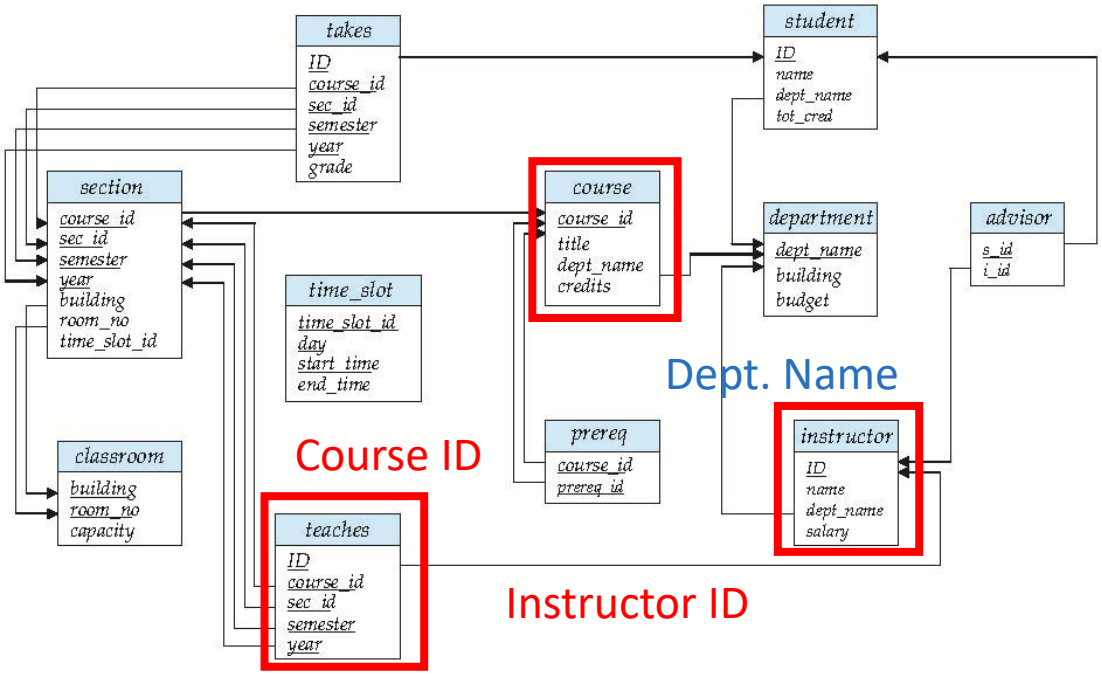
select *name, title*
from *instructor* ***natural join*** *teaches* *course*
where *teaches.course_id = course.course_id*

The relation based on the natural join and the course table are used to generated their ***Cartesian product***

Discussion



List the names of **instructors** along with the titles of **courses** that they **teach**.



select *name, title*
from *instructor* ***natural join*** *teaches, course*
where *teaches.course_id = course.course_id*

select *name, title*
from *instructor* **natural join** *teaches*
natural join *course*

Is this correct?

select *
from *instructor* **natural join** *teaches*

instructor table

teaches table

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)	course_id character varying (8)	sec_id character varying (8)	semester character varying (6)	year numeric (4)
1	10101	Srinivasan	Comp. Sci.	65000.00	CS-101	1	Fall	2009
2	10101	Srinivasan	Comp. Sci.	65000.00	CS-315	1	Spring	2010
3	10101	Srinivasan	Comp. Sci.	65000.00	CS-347	1	Fall	2009
4	12121	Wu	Finance	90000.00	FIN-201	1	Spring	2010
5	15151	Mozart	Music	40000.00	MU-199	1	Spring	2010
6	22222	Einstein	Physics	95000.00	PHY-101	1	Fall	2009
7	32343	El Said	History	60000.00	HIS-351	1	Spring	2010
8	45565	Katz	Comp. Sci.	75000.00	CS-101	1	Spring	2010
9	45565	Katz	Comp. Sci.	75000.00	CS-319	1	Spring	2010
10	76766	Crick	Biology	72000.00	BIO-101	1	Summer	2009
11	76766	Crick	Biology	72000.00	BIO-301	1	Summer	2010
12	83821	Brandt	Comp. Sci.	92000.00	CS-190	1	Spring	2009
13	83821	Brandt	Comp. Sci.	92000.00	CS-190	2	Spring	2009
14	83821	Brandt	Comp. Sci.	92000.00	CS-319	2	Spring	2010
15	98345	Kim	Elec. Eng.	80000.00	EE-181	1	Spring	2009

Data Output | Explain | Messages | Query History

	course_id character varying (8)	title character varying (50)	dept_name character varying (20)	credits numeric (2)
1	BIO-101	Intro. to Biology	Biology	4
2	BIO-301	Genetics	Biology	4
3	BIO-399	Computational Biology	Biology	3
4	CS-101	Intro. to Computer Science	Comp. Sci.	4
5	CS-190	Game Design	Comp. Sci.	4
6	CS-315	Robotics	Comp. Sci.	3
7	CS-319	Image Processing	Comp. Sci.	3
8	CS-347	Database System Concepts	Comp. Sci.	3
9	EE-181	Intro. to Digital Systems	Elec. Eng.	3
10	FIN-201	Investment Banking	Finance	3
11	HIS-351	World History	History	3
12	MU-199	Music Video Production	Music	3
13	PHY-101	Physical Principles	Physics	4

course table

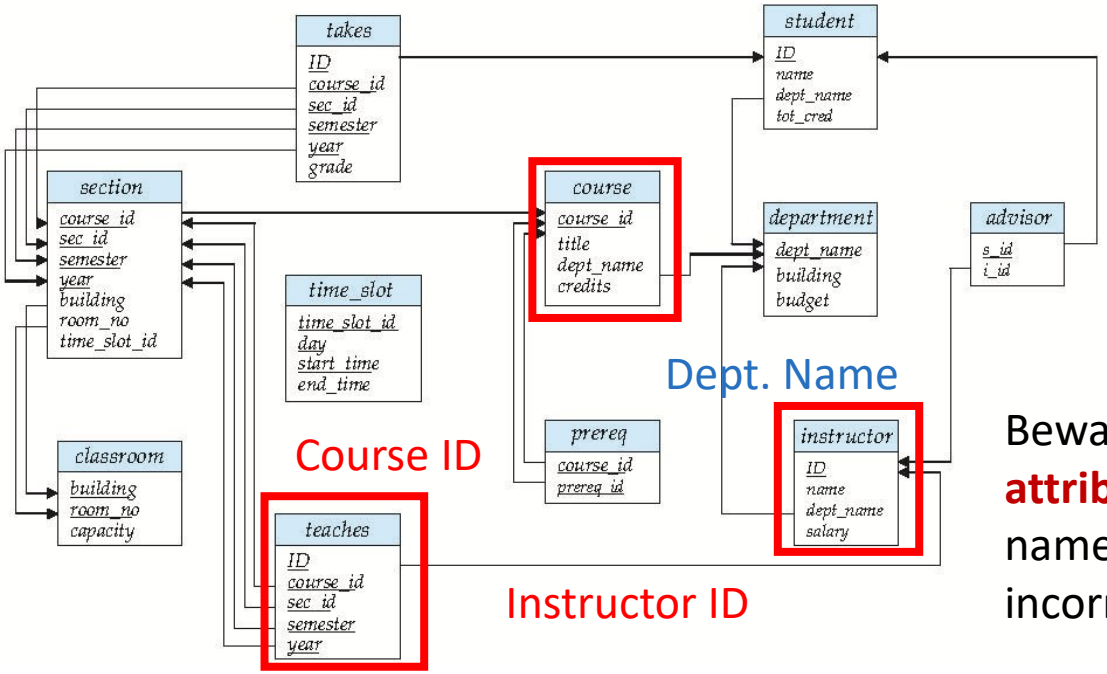
select *name, title*
from *instructor* **natural join** *teaches*
natural join *course*

select *name, title*
from *instructor, teaches, course*
where *instructor.id=teaches.id* **and**
teaches.course_id=course.course_id **and**
instructor.dept_name=course.dept_name

Discussion (Cont.)



List the names of **instructors** along with the titles of **courses** that they **teach** and offered by their own department.



Beware of **unrelated attributes** with same name which get equated incorrectly!

select *name, title*
from *instructor* **natural join** *teaches*
natural join *course*

=

select *name, title*
from *instructor, teaches, course*
where *instructor.id=teaches.id* **and**
teaches.course_id=course.course_id **and**
instructor.dept_name=course.dept_name

Data Output | Explain | Messages | Query History

	id [PK] character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)
1	10101	Srinivasan	Comp. Sci.	65000.00
2	12121	Wu	Finance	90000.00
3	15151	Mozart	Music	40000.00
4	22222	Einstein	Physics	95000.00
5	32343	El Said	History	60000.00
6	33456	Gold	Physics	87000.00
7	45565	Katz	Comp. Sci.	75000.00
8	58583	Califieri	History	62000.00
9	76543	Singh	Finance	80000.00
10	76766	Crick	Biology	72000.00
11	83821	Brandt	Comp. Sci.	92000.00
12	98345	Kim	Elec. Eng.	80000.00

The **instructor** table

Data Output | Explain | Messages | Query History

	course_id [PK] character varying (8)	title character varying (50)	dept_name character varying (20)	credits numeric (2)
1	BIO-101	Intro. to Biology	Biology	4
2	BIO-301	Genetics	Biology	4
3	BIO-399	Computational Biology	Biology	3
4	CS-101	Intro. to Computer Science	Comp. Sci.	4
5	CS-190	Game Design	Comp. Sci.	4
6	CS-315	Robotics	Comp. Sci.	3
7	CS-319	Image Processing	Comp. Sci.	3
8	CS-347	Database System Concepts	Comp. Sci.	3
9	EE-181	Intro. to Digital Systems	Elec. Eng.	3
10	FIN-201	Investment Banking	Finance	3
11	HIS-351	World History	History	3
12	MU-199	Music Video Production	Music	3
13	PHY-101	Physical Principles	Physics	4

The **course** table

Data Output | Explain | Messages | Query History

	id character varying (5)	course_id character varying (8)	sec_id character varying (8)	semester character varying (6)	year numeric (4)
1	10101	CS-101	1	Fall	2009
2	10101	CS-315	1	Spring	2010
3	10101	CS-347	1	Fall	2009
4	12121	FIN-201	1	Spring	2010
5	15151	MU-199	1	Spring	2010
6	22222	PHY-101	1	Fall	2009
7	32343	HIS-351	1	Spring	2010
8	45565	CS-101	1	Spring	2010
9	45565	CS-319	1	Spring	2010
10	76766	BIO-101	1	Summer	2009
11	76766	BIO-301	1	Summer	2010
12	83821	CS-190	1	Spring	2009
13	83821	CS-190	2	Spring	2009
14	83821	CS-319	2	Spring	2010
15	98345	EE-181	1	Spring	2009
16	98345	CS-319	1	Spring	2010

The **teaches** table

Add an extra record:
Instructor: **98345** (Kim)
Course: **CS-319** (Image Processing)

Results are different now!

Data Output | Explain | Messages | Query History

	name character varying (20)	title character varying (50)
1	Srinivasan	Intro. to Computer Science
2	Srinivasan	Robotics
3	Srinivasan	Database System Concepts
4	Wu	Investment Banking
5	Mozart	Music Video Production
6	Einstein	Physical Principles
7	El Said	World History
8	Katz	Intro. to Computer Science
9	Katz	Image Processing
10	Crick	Intro. to Biology
11	Crick	Genetics
12	Brandt	Game Design
13	Brandt	Game Design
14	Brandt	Image Processing
15	Kim	Intro. to Digital Systems

select *name, title*
from *instructor* **natural join** *teaches* **natural join** *course*

Data Output | Explain | Messages | Query History

	name character varying (20)	title character varying (50)
1	Srinivasan	Intro. to Computer Science
2	Srinivasan	Robotics
3	Srinivasan	Database System Concepts
4	Wu	Investment Banking
5	Mozart	Music Video Production
6	Einstein	Physical Principles
7	El Said	World History
8	Katz	Intro. to Computer Science
9	Katz	Image Processing
10	Crick	Intro. to Biology
11	Crick	Genetics
12	Brandt	Game Design
13	Brandt	Game Design
14	Brandt	Image Processing
15	Kim	Intro. to Digital Systems
16	Kim	Image Processing

select *name, title*
from *instructor* ***natural join*** *teaches, course*
where *teaches.course_id = course.course_id*

select *
from *instructor* **natural join** *teaches*

Data Output | Explain | Messages | Query History

	id character varying (5)	name character varying (20)	dept_name character varying (20)	salary numeric (8,2)	course_id character varying (8)	sec_id character varying (8)	semester character varying (6)	year numeric (4)
1	10101	Srinivasan	Comp. Sci.	65000.00	CS-101	1	Fall	2009
2	10101	Srinivasan	Comp. Sci.	65000.00	CS-315	1	Spring	2010
3	10101	Srinivasan	Comp. Sci.	65000.00	CS-347	1	Fall	2009
4	12121	Wu	Finance	90000.00	FIN-201	1	Spring	2010
5	15151	Mozart	Music	40000.00	MU-199	1	Spring	2010
6	22222	Einstein	Physics	95000.00	PHY-101	1	Fall	2009
7	32343	El Said	History	60000.00	HIS-351	1	Spring	2010
8	45565	Katz	Comp. Sci.	75000.00	CS-101	1	Spring	2010
9	45565	Katz	Comp. Sci.	75000.00	CS-319	1	Spring	2010
10	76766	Crick	Biology	72000.00	BIO-101	1	Summer	2009
11	76766	Crick	Biology	72000.00	BIO-301	1	Summer	2010
12	83821	Brandt	Comp. Sci.	92000.00	CS-190	1	Spring	2009
13	83821	Brandt	Comp. Sci.	92000.00	CS-190	2	Spring	2009
14	83821	Brandt	Comp. Sci.	92000.00	CS-319	2	Spring	2010
15	98345	Kim	Elec. Eng.	80000.00	EE-181	1	Spring	2009
16	98345	Kim	Elec. Eng.	80000.00	CS-319	1	Spring	2010

Data Output | Explain | Messages | Query History

	course_id character varying (8)	title character varying (50)	dept_name character varying (20)	credits numeric (2)
1	BIO-101	Intro. to Biology	Biology	4
2	BIO-301	Genetics	Biology	4
3	BIO-399	Computational Biology	Biology	3
4	CS-101	Intro. to Computer Science	Comp. Sci.	4
5	CS-190	Game Design	Comp. Sci.	4
6	CS-315	Robotics	Comp. Sci.	3
7	CS-319	Image Processing	Comp. Sci.	3
8	CS-347	Database System Concepts	Comp. Sci.	3
9	EE-181	Intro. to Digital Systems	Elec. Eng.	3
10	FIN-201	Investment Banking	Finance	3
11	HIS-351	World History	History	3
12	MU-199	Music Video Production	Music	3
13	PHY-101	Physical Principles	Physics	4

course table

select *name, title*
from *instructor* **natural join** *teaches* **natural join** *course*

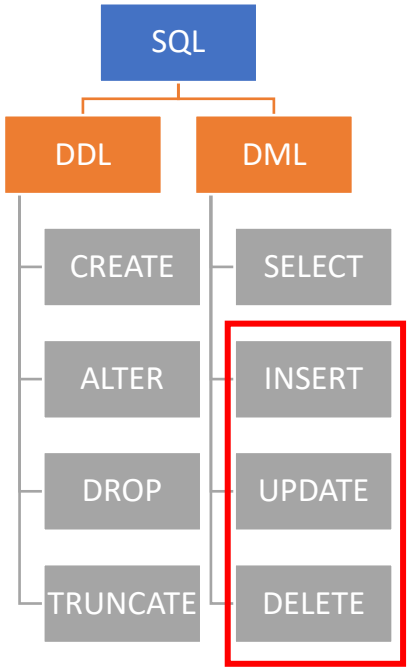
When performing the natural join, **dept_name** will also influence the join result.

In other words, in this case, those records for which an instructor teaches a course outside of his home department will be omitted.

Modification of the Database

Modification of the Database

- **Deletion** of tuples from a given relation.
- **Insertion** of new tuples into a given relation
- **Updating** of values in some tuples in a given relation



Deletion

- A delete request is expressed in much the same way as a query

delete from *r*

where *P*;

  - The delete statement first finds all tuples *t* in *r* for which *P(t)* is true, and then deletes them from *r*
  - We can only delete whole tuples

- Performing all the tests before performing any deletion

- A delete command operates on only one relation

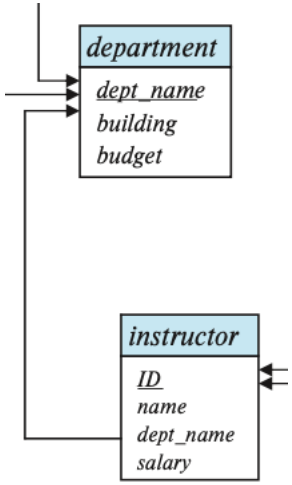
Deletion (Cont.)

- Delete all instructors:

```
delete from instructor
```

- Delete all instructors from the finance department:

```
delete from instructor
where dept_name = 'Finance'
```

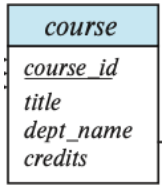


- Delete all tuples in the *instructor* relation for those instructors associated with a department located in the Watson building:

```
delete from instructor
where dept_name in (select dept_name
                    from department
                    where building = 'Waston');
```

Insertion

- Add a new tuple to *course*:



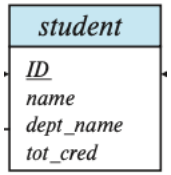
insert into *course*
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- or equivalently:

insert into *course* (*course_id*, *title*, *dept_name*, *credits*)
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- Add a new tuple to student with tot_creds set to *null*:

insert into *student*
values ('3003', 'Green', 'Finance', *null*);



Insertion (Cont.)

- Add all instructors to the student relation with tot_creds set to 0:

```
insert into student
        select ID, name, dept_name, 0
        from   instructor;
```

- The **select-from-where** statement is evaluated fully before any of its results are inserted into the relation.

```
insert into student
        select *
        from  student;
```

The insert statement would simply **duplicate** every tuple in the *student* relation if the relation did not have a primary key constraint.

Updates

- In certain situations, we may wish to change a value in a tuple without changing all values in the tuples.
- For this purpose, the **update** statement can be used.

```
update instructor
set salary = salary * 1.05;
```

- For example: Increase salaries of instructors whose salary is less than $100,000 by 5% :

```
update instructor
set salary = salary * 1.05
where salary <= 100000;
```

Updates (Cont.)

- Increase salaries of instructors whose salary is over $100,000 by 3%, and all others by 5% :

```
update instructor
set salary = salary * 1.03
where salary > 100000;

update instructor
set salary = salary * 1.05
where salary <= 100000;
```

- The order is important!

Updates (Cont.)

- SQL provides a **case** construct that we can use to perform both updates with a single **update** statement, avoiding the problem with the order of updates.

```
update instructor
set salary = case
                when salary <= 100000 then salary * 1.05
                else salary * 1.03
             end
```

- The general form of the case statement is as follows:

```
case
    when pred1 then result1
    when pred2 then result2
    …
    when predn then resultn
    else result0
end
```

Summary



- How to use nested **subqueries**?
  - *Subqueries in the where clause*
  - *Subqueries in the from clause*
  - *Subqueries in the with clause*

- Understand the usage of ***natural join***

- Know how to modify a database
  - ***delete***
  - ***insert***
  - ***update***



Thank you!

michael-yue.yu@polyu.edu.hk

Source: Google Image